

Stock Monitoring and Alerts - High-Level Design

Version: 1.0.0 | Date: 2026-05-27 | Status: Draft

Level: Enterprise | Source Plan: docs/01-plan/features/stock-monitoring-alerts.plan.md

1. Project Overview

Stock Monitoring and Alerts is a cloud-ready platform for authenticated users to manage stock watchlists, configure price alert thresholds, monitor quote trends, and receive notifications when rule conditions are met.

The platform starts with a local MVP that proves the Python monitoring engine, database persistence, mock market data, and integration contracts. The target cloud architecture introduces Oracle Visual Builder Studio / Visual Builder (VBS), OCI API Gateway, Oracle Integration Cloud (OIC), a private Python engine on an Ubuntu VM, and a database for monitoring history and audit records.

1.1 Primary Goals

- Provide authenticated dashboard access for watchlist and alert management.
- Route all public traffic through OCI API Gateway over HTTPS on port 443.
- Keep the Python engine private and inaccessible from the public internet.
- Use OIC as the orchestration brain for workflow routing and notification dispatch.
- Persist watchlists, alert targets, quote snapshots, target history, execution logs, and notification audit records.
- Use mock market data by default for local development and CI.
- Enable future live market data providers through configuration and adapter abstraction.
- Prevent secrets, credentials, provider keys, OCI wallets, and local environment files from being committed.

1.2 Non-Goals

- Trading, brokerage integration, or order placement.
- Financial advice or automated investment recommendations.
- Public direct access to the Python engine or database.
- Production live market provider integration in the initial MVP.
- Detailed REST endpoint, schema, or module-level implementation detail. Those belong in the LLD.

2. Four-Layer Architecture

The target platform is organized into four layers with strict responsibility and exposure boundaries.

Layer	Tool	Responsibility	Exposure
Front-end	Oracle Visual Builder Studio / Visual Builder	User-facing dashboard, login experience, watchlist and alert UI, graphs, history views	User-facing application only
Security and proxy	OCI API Gateway	Public HTTPS boundary, authentication, authorization, validation, routing, header propagation	Public HTTPS on port 443
Orchestration	Oracle Integration Cloud	Workflow brain, dashboard-triggered actions, alert orchestration, switch branches, notification routing	Reached through API Gateway

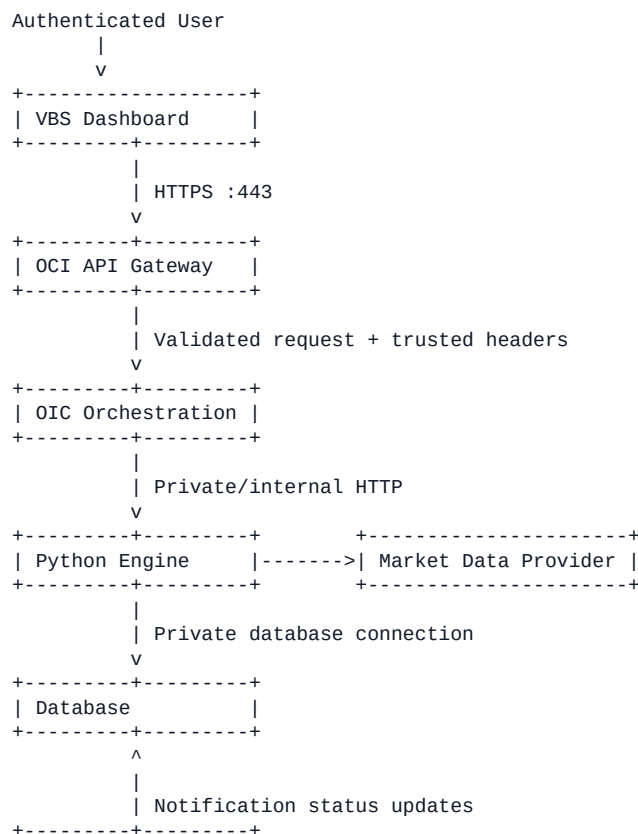
Layer	Tool	Responsibility	Exposure
Core engine and data	Ubuntu VM, Python engine, database	Monitoring logic, provider integration, alert evaluation, persistence, audit	Private/internal only

3. System Context

3.1 Actors and Systems

Actor/System	Role
Dashboard user	Authenticated user who manages symbols, thresholds, and notification preferences
VBS dashboard	User-facing dashboard for watchlists, widgets, graphs, toggles, forms, history, and notification status
OCI API Gateway	Public HTTPS ingress and policy enforcement point
Identity provider	Authentication and identity claim source for user sessions/tokens
OIC	Orchestrates dashboard commands, alert evaluations, switch branches, and notification flows
Python engine	Private service that evaluates alerts, normalizes market data, and writes persistent records
Database	Source of truth for configuration, monitoring history, execution logs, and notification audit
Market data provider	Mock provider in MVP; live provider later via configuration
Notification channels	Email, SMS, Slack, push, or simulated MVP notification audit

3.2 Context Diagram



4. Major Components

4.1 VBS Dashboard

VBS is the user-facing dashboard. It owns presentation, input forms, dashboard navigation, and user interaction patterns. It does not own orchestration logic, alert evaluation logic, backend secrets, or direct private service URLs.

Expected dashboard capabilities:

- User login and authenticated dashboard access.
- Watchlist table and stock target configuration.
- Price graph widgets and current quote summary cards.
- Alert toggle buttons and threshold forms.
- Alert history and notification status views.
- API Gateway service connections only.

4.2 OCI API Gateway

OCI API Gateway is the public security and proxy boundary. It exposes public HTTPS routes and forwards validated requests to OIC.

Major responsibilities:

- Terminate public HTTPS traffic on port 443.
- Enforce authentication and route authorization.
- Validate request method, route, and payload shape.
- Generate or propagate correlation IDs.
- Propagate trusted identity headers to OIC.
- Prevent direct public access to Python engine ports.

4.3 Oracle Integration Cloud

OIC is the orchestration brain. It coordinates user-triggered dashboard operations, alert evaluation workflows, switch branching, and notification dispatch.

Major responsibilities:

- Receive API Gateway requests.
- Validate workflow-level preconditions.
- Invoke private Python engine endpoints.
- Route success, validation, provider error, and system error branches.
- Route notification channel branches.

- Update notification audit status after downstream delivery.
- Preserve correlation IDs across flows.

4.4 Python Engine on Ubuntu VM

The Python engine is a private internal service. It owns domain behavior and persistence-facing logic.

Major responsibilities:

- Run stock monitoring and alert evaluation logic.
- Integrate with market data providers through a provider abstraction.
- Use mock market data by default for local development and CI.
- Persist watchlists, alert targets, quote snapshots, target history, execution logs, and notification audit records.
- Expose internal APIs only for OIC/API Gateway integration paths.
- Emit structured logs with correlation IDs.

4.5 Database

The database is the source of truth for product configuration and auditability.

Major responsibilities:

- Store user references and watchlist configuration.
- Store alert targets and target change history.
- Store quote snapshots for graphs, trend views, and evaluation history.
- Store execution logs for operational traceability.
- Store notification audit records and downstream delivery status.
- Support retention and backup policies for production use.

5. Public and Private Network Boundaries

5.1 Public Boundary

- Public access is allowed only through OCI API Gateway over HTTPS on port 443.
- VBS calls API Gateway routes only.
- No Python engine port is exposed publicly.
- No database endpoint is exposed publicly.

5.2 Private Boundary

- OIC invokes Python engine APIs through private/internal HTTP connectivity.
- Python engine connects to the database through a private database connection.
- Python engine may make controlled outbound HTTPS requests to market data providers.

- Notification delivery is orchestrated through OIC connectors or downstream services.

5.3 Boundary Enforcement Principles

- API Gateway is the trust boundary for public authentication and authorization.
- OIC is trusted only through configured API Gateway and private connectivity.
- Python engine trusts propagated identity context only from trusted internal paths.
- Secrets are injected by environment or secret management, never committed to Git.

6. Required Data Flow

The target flow is:

```
VBS Dashboard
-> OCI API Gateway over HTTPS :443
-> OIC orchestration
-> Python Engine on Ubuntu VM over private/internal HTTP
-> Database
-> Market Data Provider
-> OIC notification flow
```

Flow interpretation:

- Dashboard users interact only with VBS.
- VBS sends API calls only to OCI API Gateway.
- API Gateway authenticates, validates, and proxies to OIC.
- OIC decides workflow routing and invokes the private engine.
- Python engine evaluates domain logic and persists state.
- Market data is accessed through provider adapters.
- Triggered notifications flow back through OIC for channel dispatch and audit updates.

7. VBS Dashboard Flow

```
User signs in
-> VBS loads dashboard shell
-> VBS requests watchlist and alert state through API Gateway
-> User adds or updates watchlist symbol
-> VBS submits form to API Gateway
-> API Gateway forwards validated request to OIC
-> OIC invokes Python engine
-> Engine persists change and returns result
-> OIC maps response
-> VBS refreshes bound dashboard data
```

Dashboard expectations:

- Forms validate required values before submit.
- Toggle state is refreshed from the backend after command completion.
- Graphs render persisted quote snapshots.
- Notification status views reflect audit records updated by OIC.

8. OCI API Gateway Authentication and Proxy Flow

```
HTTPS request arrives at API Gateway
-> TLS termination
-> Token/session validation
-> Route authorization
-> Request validation
-> Correlation ID generation or propagation
-> Trusted identity header mapping
-> Request forwarding to OIC integration endpoint
-> Response returned to VBS
```

Gateway responsibilities:

- Public TLS boundary.
- Authentication and authorization policy enforcement.
- Public route definitions.
- Payload and header controls.
- Identity and correlation propagation.
- Routing only to approved OIC targets.

9. OIC Orchestration Flow

```
OIC receives request from API Gateway
-> Validate required headers and workflow inputs
-> Normalize request envelope
-> Invoke Python engine private endpoint
-> Evaluate response outcome
-> Switch branch:
    success
    no alert triggered
    alert triggered
    validation error
    provider error
    system error
-> Dispatch notifications when needed
-> Patch notification audit status
-> Return normalized response to API Gateway
```

OIC owns sequencing and integration behavior. Python owns alert business rules.

10. Python Engine Responsibilities

The Python engine remains private and focused on core domain behavior.

Responsibilities:

- Normalize input from OIC.
- Scope user data to authenticated identity context.
- Manage watchlist and alert target state.
- Retrieve quotes through configured market data provider adapter.
- Evaluate alert rules.
- Suppress uncontrolled duplicate notifications.

- Write quote snapshots, target history, execution logs, and notification audit rows.
- Return deterministic responses for OIC mapping.
- Provide health and readiness checks for internal operations.

11. Database Responsibilities

The database stores monitoring history and audit records from the start.

Persistence areas:

- User references.
- Watchlists.
- Alert targets.
- Quote snapshots.
- Target history.
- Execution logs.
- Notification audit records.
- Provider configuration metadata where appropriate.

Operational expectations:

- Use migrations from the first implementation phase.
- Support deterministic local seed data.
- Support indexes for user, symbol, status, and timestamp access patterns.
- Preserve audit records even when watchlist or alert targets are disabled or soft deleted.

12. Market Data Provider Integration Strategy

12.1 MVP Strategy

- Mock provider is the default provider for local development and CI.
- Mock data must be deterministic.
- Mock scenarios should cover trigger, no-trigger, unknown symbol, stale data, and provider failure cases.
- CI must not require live provider credentials or internet access.

12.2 Future Live Provider Strategy

- Live providers are added through adapter implementations.
- Provider selection is environment/configuration driven.
- Provider credentials come from environment variables or secret management.
- Provider-specific responses are normalized before alert evaluation.
- Rate limits, retries, timeouts, and provider-specific failures are handled in the adapter layer.

13. Notification Architecture

13.1 MVP Notification Behavior

- Python engine records notification intent in notification audit records.
- Local development may mark notifications as simulated.
- No production email, SMS, Slack, or push delivery is required for MVP.

13.2 Target Notification Behavior

```
Alert is triggered
-> Engine creates notification audit record
-> OIC receives notification item
-> OIC Switch selects channel
-> Channel connector sends email, SMS, Slack, or push
-> OIC updates notification audit status
-> Dashboard displays delivery state
```

Notification states:

- pending
- simulated
- sent
- skipped
- failed
- retry scheduled

14. Security Model

14.1 Authentication and Authorization

- VBS and API Gateway integrate with the configured identity provider.
- API Gateway validates public identity tokens or sessions.
- API Gateway enforces route-level authorization.
- OIC validates workflow-level requirements.
- Python validates trusted user context for user-scoped data.

14.2 Network Security

- API Gateway is the only public backend ingress.
- Python engine is private/internal only.
- Database is private and reachable only by approved backend paths.
- Outbound provider access is controlled and auditable.

14.3 Secret Management

- `.env` and `.env.*` files are ignored by Git.
- `.env.example` may contain non-secret placeholders.
- Provider API keys, OCI keys, wallets, certificates, and Terraform variable files are never committed.
- Production secrets should be injected by environment or secret management.

14.4 Auditability

- Alert evaluations create execution records.
- Triggered notifications create audit records.
- Notification delivery status is updated by OIC.
- Correlation IDs connect API Gateway, OIC, engine, and database records.

15. Operations and Observability Model

15.1 Logging

- API Gateway logs public route activity.
- OIC records integration instance history and branch outcomes.
- Python emits structured logs with correlation IDs.
- Provider and notification failures include normalized error codes.

15.2 Metrics

Target metrics:

- API request count and error count.
- Quote refresh success/failure count.
- Alert evaluation count.
- Triggered alert count.
- Notification count by channel and status.
- Provider latency and failure count.
- Database connectivity and operation failure count.

15.3 Health and Readiness

- Python health check confirms process liveness.
- Python readiness check confirms database and provider configuration readiness.
- API Gateway route smoke tests verify public route health.
- OIC monitoring verifies integration health and failed branches.

15.4 Backup and Retention

- Production database requires backup and recovery procedures.
- Quote snapshots may use retention or aggregation policies.
- Audit records should be retained long enough for support and compliance review.

16. Deployment View

16.1 MVP Deployment

Developer workstation or CI runner

- > Python engine
- > Local database
- > Mock provider
- > Simulated notification audit

MVP deployment validates the engine, persistence, mock provider, and contracts without requiring cloud provisioning.

16.2 Target Cloud Deployment

VBS hosted dashboard

- > OCI API Gateway public HTTPS endpoint
- > OIC integration instance
- > Private Ubuntu VM running Python engine
- > Private database
- > External market data provider over controlled HTTPS egress
- > OIC notification connectors

16.3 Deployment Controls

- Environment-specific configuration.
- Database migrations before engine rollout.
- No committed secrets.
- Health and smoke checks after deployment.
- Rollback plan for engine release and configuration changes.

17. MVP Scope Versus Planned Cloud Integration Scope

Capability	MVP	Planned Cloud Integration
Dashboard	API contracts and sample payload assumptions	Full VBS dashboard screens and bindings
Gateway	Local route shape only	OCI API Gateway HTTPS, auth, route policies
OIC	Flow design and integration assumptions	OIC flows, switch branches, notification orchestration
Engine	Python service with mock provider	Private VM deployment
Database	Local/dev persistence and migrations	Production database, backup, retention
Market data	Mock provider	Live provider adapter
Notifications	Simulated audit rows	OIC email, SMS, Slack, push delivery
Operations	Local logs and smoke tests	OCI logging, monitoring, runbooks

18. Risks and Mitigations

Risk	Impact	Mitigation
Python engine is accidentally exposed	High	Keep public ingress only at API Gateway; use private subnet/security rules
OIC accumulates business logic	Medium	Keep alert evaluation and duplicate suppression inside Python
Mock provider hides live provider edge cases	Medium	Add provider adapter contract tests and mock failure scenarios
Duplicate notifications annoy users	Medium	Persist alert state and notification audit before channel dispatch
Provider limits block live rollout	High	Evaluate provider terms early and isolate provider choice behind configuration
Database history grows quickly	Medium	Plan retention, indexes, and later aggregation
Secrets leak into repo	High	Use .gitignore, .env.example, environment injection, and review checks
Dashboard assumes direct backend access	High	VBS service connections must target API Gateway only

19. HLD Acceptance Checklist

- [x] Project overview is defined.
- [x] Four-layer architecture is documented.
- [x] System context is documented.
- [x] Major components are documented.
- [x] Public/private network boundaries are documented.
- [x] Required data flow is documented.
- [x] VBS dashboard flow is documented.
- [x] OCI API Gateway authentication and proxy flow is documented.
- [x] OIC orchestration flow is documented.
- [x] Python engine responsibilities are documented.
- [x] Database responsibilities are documented.
- [x] Market data provider integration strategy is documented.
- [x] Notification architecture is documented.
- [x] Security model is documented.
- [x] Operations and observability model is documented.
- [x] Deployment view is documented.
- [x] MVP scope versus planned cloud integration scope is documented.
- [x] Risks and mitigations are documented.
- [x] Document stays HLD-only and avoids LLD endpoint/schema/module details.